

Лабораторная работа №3. HA Postgres Cluster

Вариант 2

Цель работы:

Развертывание и настройка высокодоступного кластера PostgreSQL с использованием Patroni, ZooKeeper и HAProxy

Для выполнения лабораторной работы была создана отдельная директория проекта., и были подготовлены конфигурационные файлы Docker, Patroni и HAProxy

В директории проекта находятся следующие файлы:

Dockerfile;
docker-compose.yml;
postgres0.yml;
postgres1.yml;
haproxy.cfg

```
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~$ cd lab3AdmCompSeti
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ ls -l
итого 20
-rw-rw-r-- 1 enfantsrichesdepression docker 1356 сен 27 18:47 docker-compose.yml
-rw-rw-r-- 1 enfantsrichesdepression docker 442 сен 25 20:45 Dockerfile
drwxr-xr-x 2 root root 4096 сен 27 18:44 haproxy.cfg
-rw-rw-r-- 1 enfantsrichesdepression docker 1320 сен 25 20:51 postgres0.yml
-rw-rw-r-- 1 enfantsrichesdepression docker 1318 сен 25 21:46 postgres1.yml
```

1. Подготовка PostgreSQL и Patroni

Для запуска PostgreSQL совместно с Patroni был подготовлен Docker образ на основе PostgreSQL 15

В образ были установлены необходимые зависимости, Python и Patroni. Также в образ копируются конфигурационные файлы двух нод PostgreSQL

```

enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ cat Dockerfile
FROM postgres:15

RUN apt-get update -y && \
    apt-get install -y netcat-openbsd python3-pip curl python3-psycpg2 python3-
venv iputils-ping

RUN python3 -m venv /opt/patroni-venv && \
    /opt/patroni-venv/bin/pip install --upgrade pip && \
    /opt/patroni-venv/bin/pip install patroni[zookeeper] psycpg2-binary

COPY postgres0.yml /postgres0.yml
COPY postgres1.yml /postgres1.yml

ENV PATH="/opt/patroni-venv/bin:$PATH"

USER postgres

```

Для первой ноды кластера был создан файл postgres0.yml

```

1xxx:~/lab3AdmCompSeti$ cat postgres0.yml
scope: my_cluster
name: postgresql0

restapi:
  listen: pg-master:8008
  connect_address: pg-master:8008

zookeeper:
  hosts:
    - zoo:2181

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 10485760
    master_start_timeout: 300
    synchronous_mode: true

```

Аналогичным образом был создан файл postgres1.yml для второй ноды

```

1xxx:~/lab3AdmCompSeti$ cat postgres1.yml
scope: my_cluster
name: postgresql1

restapi:
  listen: pg-slave:8008
  connect_address: pg-slave:8008

zookeeper:
  hosts:
    - zoo:2181

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 10485760
    master_start_timeout: 300
    synchronous_mode: true

```

Для запуска компонентов был создан файл docker-compose.yml
 В нём были описаны две ноды PostgreSQL, ZooKeeper и HAProxy. Контейнер pg-master доступен через порт 5433, pg-slave - через порт 5434, а HAProxy используется внешний порт 5435

```

enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ cat docker-compose.yml
services:
  pg-master:
    build: .
    image: localhost/postres:patroni
    container_name: pg-master
    restart: always
    hostname: pg-master
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      PGDATA: '/var/lib/postgresql/data/pgdata'
    expose:
      - 8008
    ports:
      - "5433:5432"
    volumes:
      - pg-master:/var/lib/postgresql/data
    command: patroni /postgres0.yml

```

После подготовки конфигурационных файлов кластер был запущен с помощью Docker Compose

Для проверки состояния контейнеров была выполнена команда **docker ps**. Все необходимые компоненты успешно запустились: две ноды PostgreSQL, ZooKeeper и HAProxy

```
✓ localhost/postres:patroni Built 0.0s
✓ Container pg-master Running 0.0s
✓ Container pg-slave Running 0.0s
✓ Container zoo Running 0.0s
✓ Container postgres_entrypoint Start... 0.2s
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATE
ID            STATUS      PORTS
NAMES
e69e9af9dea5   localhost/postres:patroni          "docker-entrypoint.s..." 56 min
utes ago      Up 56 minutes   8008/tcp, 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
pg-slave
f8c9b01a92b4   localhost/postres:patroni          "docker-entrypoint.s..." 56 min
utes ago      Up 56 minutes   8008/tcp, 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pg-master
c050ee0092dc   confluentinc/cp-zookeeper:7.7.1   "/etc/confluent/dock..." 46 hou
rs ago        Up 59 minutes   2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
, 3888/tcp    zoo
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$
```

После запуска было проверено состояние кластера с помощью утилиты patronictl. Нода postgresql0 получила роль Leader, а postgresql1 работает как Sync Standby, обе ноды находятся в рабочем состоянии, задержка репликации равна 0

```
1xxx:~/lab3AdmCompSeti$ docker exec pg-master patronictl -c /postgresql/ha-test
+ Cluster: my_cluster (7689547306326052897) -----+-----+-----+-----+
+-----+-----+
| Member      | Host      | Role      | State      | TL | Receive LSN | Lag |
Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
| postgresql0 | pg-master | Leader     | running    | 3 |              |      |
|              |          |            |            |   |              |      |
| postgresql1 | pg-slave  | Sync Standby | streaming  | 3 | 0/60000000 | 0 |
0/60000000 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$
```

2. Проверка репликации

Для проверки работы репликации в ведущей ноде была создана тестовая таблица lab3_test, в нее были добавлены две тестовые записи. После выполнения SELECT данные успешно отображались в ведущей ноде

```
postgres=# SELECT * FROM lab3_test;
 id |      message
-----+-----
  1 | First row
  2 | Replication test
(2 rows)

postgres=#
```

После этого было выполнено подключение ко второй ноде pg-slave. Созданная на ведущей ноде таблица и ее данные автоматически появились на резервной ноде, что подтверждает работу потоковой репликации
Попытка выполнить INSERT на резервной ноде завершилась ошибкой, это подтверждает, что нода работает в режиме реплики только для чтения

```
postgres=# SELECT * FROM lab3_test;
 id |      message
-----+-----
  1 | First row
  2 | Replication test
(2 rows)

postgres=# INSERT INTO lab3_test (message)
VALUES ('Should fail');
ERROR:  cannot execute INSERT in a read-only transaction
postgres=#
```

После подготовки конфигурационных файлов был выполнен запуск всех компонентов кластера с помощью Docker Compose, для проверки состояния контейнеров была выполнена команда docker ps
В результате были успешно запущены две ноды PostgreSQL, ZooKeeper и HAProxy. Контейнер postgres_entrypoint используется как единая точка подключения к кластеру PostgreSQL.

```

enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATE
D             STATUS    PORTS                               NAMES
6f308ded64c5   haproxy:3.0                        "docker-entrypoint.s..."            30 sec
onds ago      Up 30 seconds    0.0.0.0:7000->7000/tcp, [::]:7000->7000/tcp, 0.0.0.0:
5435->5432/tcp, [::]:5435->5432/tcp    postgres_entrypoint
87948dc6992e   confluentinc/cp-zookeeper:7.7.1   "/etc/confluent/dock..."            4 minu
tes ago      Up 4 minutes     2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
, 3888/tcp    zoo
217baf2fde82   localhost/postres:patroni          "docker-entrypoint.s..."            4 minu
tes ago      Up 4 minutes     8008/tcp, 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
pg-slave
6d3e00310937   localhost/postres:patroni          "docker-entrypoint.s..."            4 minu
tes ago      Up 4 minutes     8008/tcp, 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pg-master

```

Для проверки состояния кластера была использована утилита patronictl. После запуска обе ноды PostgreSQL находились в рабочем состоянии: нода postgresql1 выполняла роль Leader, а postgresql0 работала как резервная нода

```

+ Cluster: my_cluster (7689547306326052897) -----+-----+-----+-----+
-----+-----+
| Member      | Host      | Role          | State   | TL | Receive LSN | Lag |
Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
| postgresql0 | pg-master | Sync Standby | streaming | 4 | 0/90000C0 | 0 |
0/90000C0 | 0 |
| postgresql1 | pg-slave  | Leader       | running  | 4 |           |   |
|             |           |              |          |   |           |   |
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+

```

3. Проверка работы HAProxy

Для организации единой точки подключения к кластеру был настроен HAProxy. После запуска было выполнено подключение к PostgreSQL через сервис haproxy. Команда `SELECT pg_is_in_recovery();` вернула значение `false`, что подтверждает подключение к текущей ведущей ноде

Также через HAProxy были успешно получены данные из таблицы `lab3_test`

```

1xxx:~/lab3AdmCompSeti$ docker exec pg-slave getent hosts haproxy
172.19.0.5      haproxy
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker exec -e PGPASSWORD=postgres -it pg-slave \
psql -h haproxy -p 5432 -U postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
 pg_is_in_recovery 
-----
 f
(1 row)

postgres=# SELECT * FROM lab3_test;
 id |      message
-----+-----
  1 | First row
  2 | Replication test

```

4. Проверка автоматического восстановления ноды

В качестве индивидуального задания был выбран вариант, при котором после возвращения отключённой ноды в кластер она должна автоматически получить данные, записанные во время её отсутствия

Для моделирования временной недоступности резервная нода pg-master была принудительно остановлена. После остановки Leader, ZooKeeper и HAProxy продолжили работу

```

enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker stop pg-master
pg-master
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATE
D             STATUS                PORTS                    NAMES
6f308ded64c5   haproxy:3.0              "docker-entrypoint.s..." 22 min
utes ago      Up 22 minutes         0.0.0.0:7000->7000/tcp, [::]:7000->7000/tcp, 0.0.0.0:
5435->5432/tcp, [::]:5435->5432/tcp   postgres_entrypoint
87948dc6992e   confluentinc/cp-zookeeper:7.7.1   "/etc/confluent/dock..." 27 min
utes ago      Up 27 minutes         2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
, 3888/tcp                                         zoo
217baf2fde82   localhost/postgres:patroni        "docker-entrypoint.s..." 27 min
utes ago      Up 27 minutes         8008/tcp, 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
pg-slave

```

Пока резервная нода pg-master была недоступна, через HAProxy была выполнена запись новых данных в таблицу lab3_test, операция выполнена успешно, несмотря на отсутствие второй ноды

```

enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es
1xxx:~/lab3AdmCompSeti$ docker exec -e PGPASSWORD=postgres -it pg-slave \
psql -h haproxy -p 5432 -U postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# INSERT INTO lab3_test (message)
VALUES ('Created while replica was offline');
INSERT 0 1
postgres=# SELECT * FROM lab3_test;
 id |
-----+-----
  1 | First row
  2 | Replication test
  3 | Created while replica was offline
(3 rows)

```

После добавления новых данных резервная нода pg-master была снова запущена. Через некоторое время Patroni автоматически вернул её в кластер в качестве резервной ноды. Состояние streaming показывает, что репликация с ведущей нодой восстановлена

```

+ Cluster: my_cluster (7689547306326052897) ----+-----+-----+-----+-----+-----+
+-----+-----+
| Member      | Host      | Role    | State    | TL | Receive LSN | Lag | Repl   |
| y LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
| postgresql0 | pg-master | Replica | streaming | 4 | 0/902D5D8 | 0 | 0/902D5D8 | 0 |
| postgresql1 | pg-slave  | Leader  | running   | 4 |           |   |           |   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
enfantsrichesdepression@enfantsrichesdepression-HP-Envy-x360-2-in-1-Laptop-14-es

```

После восстановления репликации было выполнено прямое подключение к вернувшейся ноде pg-master. Команда SELECT pg_is_in_recovery(); вернула значение true, что подтверждает работу данной ноды в режиме реплики. После выполнения SELECT из таблицы lab3_test была выведена запись созданная в момент, когда pg-master была отключена

Вывод:

В ходе лабораторной работы был развёрнут высокодоступный кластер PostgreSQL с использованием Patroni, ZooKeeper и HAProxy. Были настроены две ноды PostgreSQL, между которыми была проверена потоковая репликация. Одна нода выполняла роль Leader, а вторая работала как резервная.

HAProxy был настроен как единая точка подключения к кластеру и корректно направлял соединения на текущую ведущую ноду.

В рамках второго варианта резервная нода была временно отключена. Во время её отсутствия в базу данных была добавлена новая запись. После повторного запуска нода автоматически вернулась в кластер, восстановила репликацию и получила данные, записанные во время её отсутствия